PINESPHERE STAY

Device Management Module

Engineering Blueprint & Implementation Plan

PRODUCT	Pinesphere Stay — Offline-First Property Management SaaS
MODULE OWNER	Sri — Middleware Developer, Pinesphere Solutions
MODULE SCOPE	Device Management (registration, licensing binding, approval, remote control, sync health, dashboards)
TECH STACK	Flutter (mobile) • FastAPI (backend) • PostgreSQL (central) • SQLite / ObjectBox (on-device)
SOURCE	Derived entirely from Pinesphere Stay – Business Requirement Document

Table of Contents

1. Module Overview & Scope

Pinesphere Stay is an **offline-first** Property Management SaaS: the owner's Android app must keep working with zero internet connectivity, and the platform enforces subscriptions through a signed, locally verifiable license token rather than a live login check. Every one of those guarantees is anchored on a single physical unit — the **device**. The Device Management module is therefore the trust boundary of the whole platform: it decides which physical phones/tablets are allowed to hold a property's offline data and license, tracks whether each device is healthy and in sync, and gives Pinesphere and property owners a way to instantly cut off a device that should no longer have access.

Per the BRD, this module must directly deliver:

- **Device registration** — every mobile must be registered before it can operate the app.
- **Device directory fields** — Device ID, Android Version, IMEI/Android ID, Last Sync, Last Login, Current User, Battery, Location.
- **Device actions** — Approve, Reject, Lock, Logout, Disable, Rename, Transfer.
- **Remote device control** — revoke license, force logout, erase local data on reconnect, disable login (staff-resignation flow).
- **Binding to the Offline License System** — license tokens are generated per Device ID + Property ID, downloaded once, and verified locally with a digital signature, with no internet required afterwards.
- **Device-count enforcement** — each Subscription Plan defines a Maximum Device count that this module must police.
- **Sync visibility** — Online/Offline status, Sync Status, Failed Syncs feed the Super Admin dashboard and the platform's Notification Center (“Device Offline”, “Sync Failure”).
- **Support tooling** — Force Sync and Remote Diagnostics for the Support Portal.
- **Auditability** — every device action must be written to the immutable Audit Log.

Out of scope for this module (owned elsewhere but referenced): Subscription/plan purchase and billing (Subscription Management), the actual booking/guest data that syncs (Booking, Guest, Housekeeping modules), and creation of staff accounts themselves (Staff Management / User & Role Management) — Device Management only manages the hardware unit a staff member logs in from.

2. Where Device Management Fits in the Platform

The BRD frames the SaaS platform as three applications sharing one multi-tenant FastAPI backend: the Pinesphere Admin Portal (Super Admin), the Property Owner App, and the Guest Portal. Device Management is consumed from two of these three surfaces, plus the internal Support Portal:

- **Super Admin (Pinesphere Operations)** — global visibility and control over every device across every tenant property: approve/reject, lock/disable, revoke, and view device/sync analytics on their dashboard.
- **Property Owner** — per the BRD's Owner Login capabilities (“...Manage Devices...”), the owner manages devices scoped only to their own property — approving their own staff's phones and revoking access when staff leave.
- **Support Portal** — support engineers get narrower, diagnostic-only access: Force Sync, Remote Diagnostics, View Sync Status, View Errors — without the ability to revoke a license or change a subscription.

Device Management sits directly beside — and is data-linked to — three other modules: **Subscription Management** (device count ceilings, license token generation, grace-period rules), **User & Role Management / Staff Management** (who is currently logged in on a device), and the **Synchronization Engine** (last sync timestamp, failed-sync tracking, background retry). All of these are modelled as foreign-key relationships in Section 5.

3. Core Business Logic & Rules

3.1 Device Registration & Activation

A new device can only be activated once, and the very first activation for a device requires internet connectivity (a license cannot be minted without contacting the server at least once).

- The staff/owner enters the Property ID and an activation code (shared during onboarding / WhatsApp welcome message).
- The app derives a stable **Device UID** from the Android ID (or IMEI where available) and hashes it before transmission.
- The device is created server-side with status = pending_approval unless auto-approval rules apply (e.g. the property's very first device, or an owner-configured auto-approve toggle).
- The server checks the property's current active device count against subscription_plans.max_devices before allowing approval — this is the same limit referenced in the BRD's Subscription Management plan matrix.
- On approval, the server mints an **encrypted License Token** containing Device UID + Property ID + Expiry Date + Digital Signature (identical structure to the BRD's Offline License System), and pushes it to the device.
- The token is stored locally (encrypted) and verified using the stored public key on every app start — no server call is required afterwards.

3.2 Device-Count Enforcement

Every Subscription Plan (Starter, Basic, Standard, Professional, Enterprise, or a Custom/Corporate plan) carries a max_devices ceiling. Before any device moves to active, the backend re-validates the count of non-revoked devices for that property. If the property is already at its limit, the approval is blocked and the owner is prompted to either deactivate an existing device or upgrade the plan (a deep link into Subscription Management, not duplicated in this module).

3.3 Local (Offline) License Validation

On every app launch and periodically in the background, the Flutter app reads its local license record and: (1) verifies the digital signature against the embedded public key, (2) confirms the Device UID and Property ID inside the token match the current device, and (3) compares the expiry date against system time. The resulting status — *active*, *grace* (per the BRD's 7-day grace period), or *suspended* — gates local feature availability (e.g. new bookings and check-in are disabled from Day 8 without renewal, while reports remain view-only and export stays available, exactly as specified in the BRD's Non-Payment Flow).

3.4 Device Actions (Owner / Super Admin)

Action	Effect
Approve	Moves device from pending_approval → active; triggers license token generation.
Reject	Moves device to rejected (terminal); no token issued; staff notified to contact owner.
Lock	Temporary block — device can be unlocked again without re-registration. Local app shows a lock screen; no data access.
Logout	Force-ends the current device session; user must sign back in (device stays registered).
Disable	Soft, longer-term block; identical effect to Lock but intended for longer suspensions (e.g. staff on leave).
Rename	Cosmetic-only change to the device's display label in the directory; no effect on license or access.
Transfer	Reassigns a device's primary_user to a different staff member without re-registering the physical device — used when a phone is handed to a new employee.
Revoke License	Invalidates the license token server-side; on next reconnect the device force-logs-out, erases its local database, and disables login until re-registered.

3.5 Remote Device Control (e.g. Staff Resignation)

This is the BRD's explicit “If staff resign” flow: Super Admin (or Owner) selects the device → Revoke License → Logout → Erase Local Database when device reconnects → Disable Login. Because the device may be offline at the moment of revocation, the command is **queued server-side** and delivered the next time the device performs a sync check-in — the device cannot be reached directly, so the design must not assume push delivery is guaranteed and must re-check revoke status on every sync attempt as a fallback.

3.6 Sync Health Monitoring

Each sync attempt (success, partial, or failure) is recorded. A device is flagged *Offline* on dashboards when last_sync_at exceeds a configurable threshold, feeding the Notification Center's “Device Offline” and “Sync Failure” alerts. Failed syncs trigger the Synchronization Engine's Auto-Retry (exponential backoff) and are resumable after interruption, per the BRD's stated sync features.

3.7 Audit Logging

Every device action (approve, reject, lock, unlock, logout, disable, enable, rename, transfer, revoke) is written to an append-only log, mirrored into the platform-wide Audit Log described in the BRD (“Nothing can be deleted from the audit log”). Each entry captures who performed the action, their role, a reason where applicable, and a timestamp.

4. Workflows

4.1 Device Registration & Activation Flow

Diagram 1 — Device Registration & Activation Flow

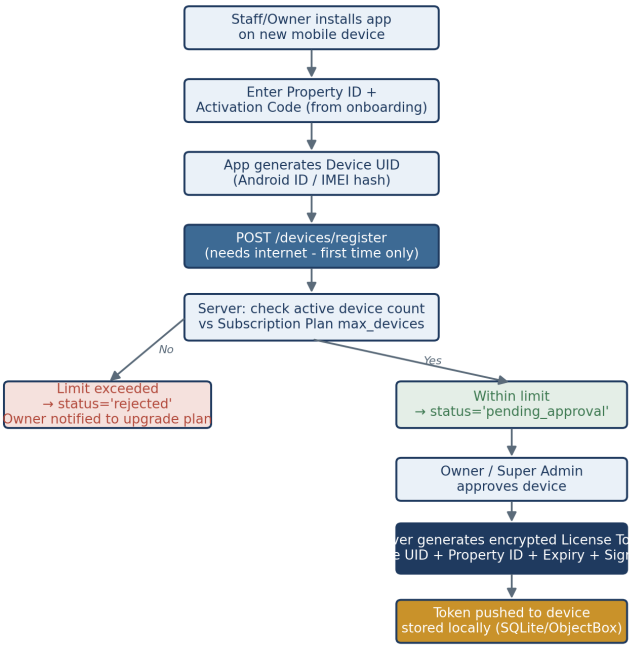

Diagram 1 — First-time device registration through to a locally stored license token.

4.2 Device Lifecycle State Machine

Diagram 2 — Device Lifecycle State Machine

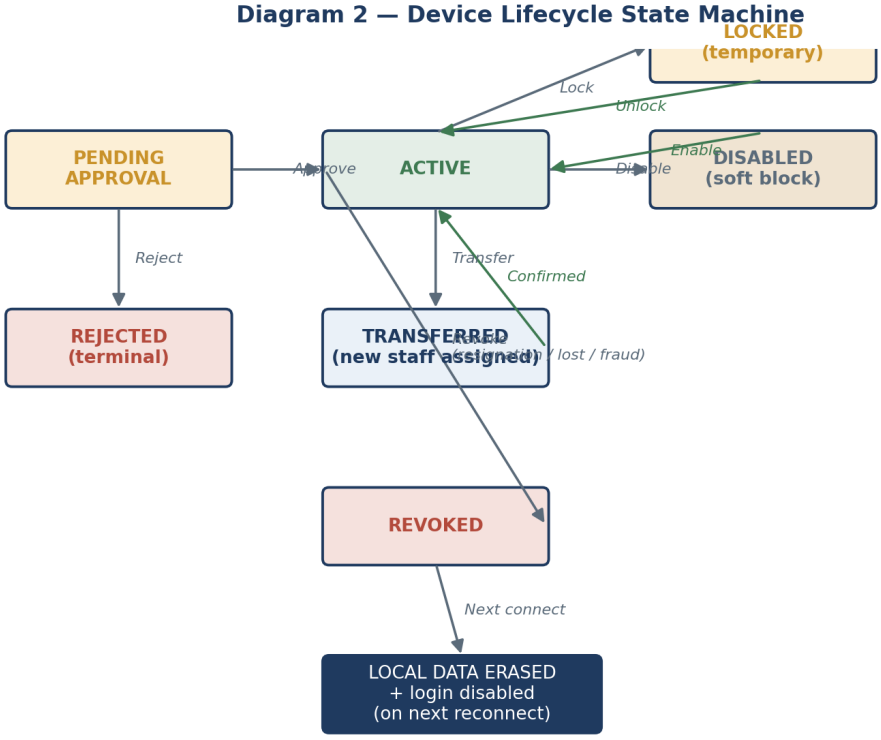

Diagram 2 — All device states and the actions that transition between them.

4.3 Remote Device Deactivation (Staff Resignation)

Diagram 3 — Remote Device Deactivation (e.g. Staff Resignation)

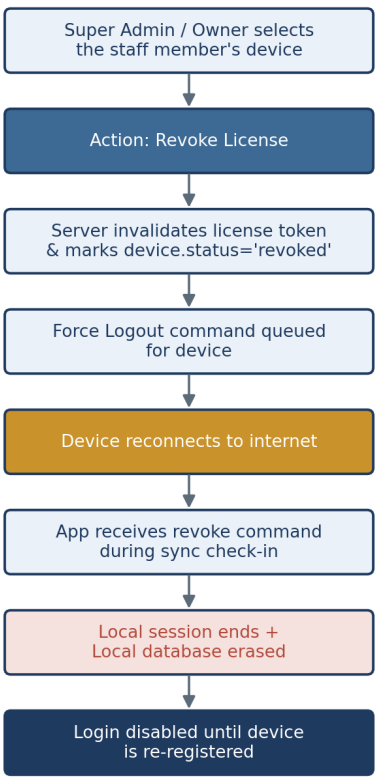

Diagram 3 — Exact flow specified in the BRD for revoking a resigned staff member's device.

4.4 Sync Status Detection & Offline Alerting

Diagram 4 — Sync Status Detection & Offline Alerting

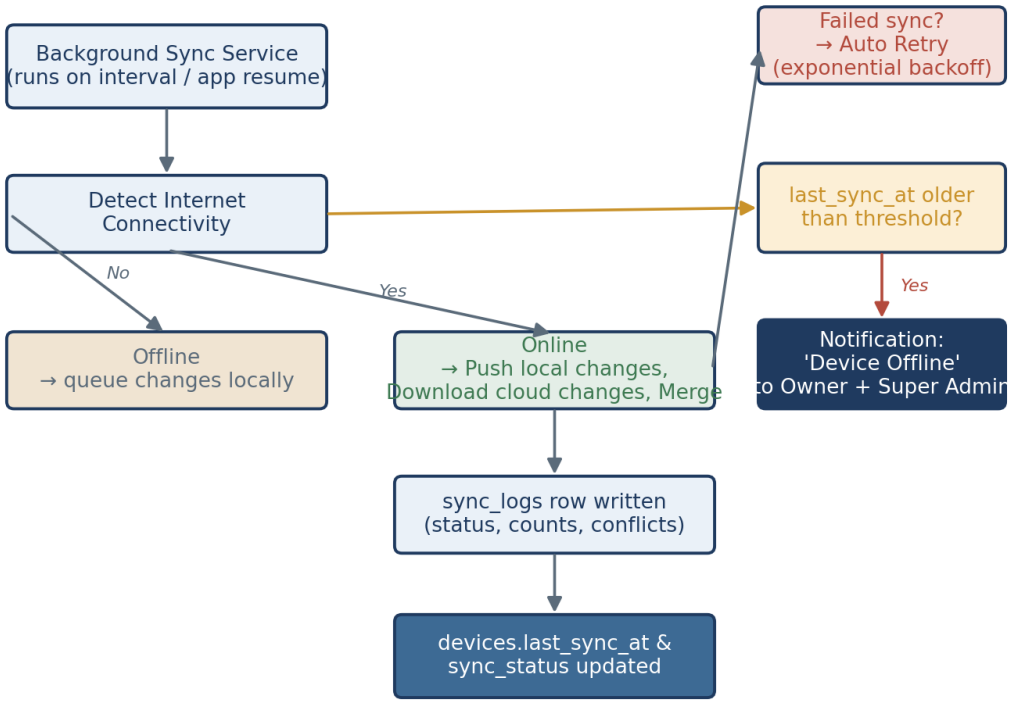

Diagram 4 — How sync attempts translate into dashboard status and notifications.

5. Database Design

The module spans two physical data stores, matching the stated tech stack: a central multi-tenant **PostgreSQL** database (owned by FastAPI) that gives Super Admin and Owners their global/scoped views, and a minimal **on-device SQLite/ObjectBox** store that lets the Flutter app validate its own license completely offline.

5.1 Entity-Relationship Diagram (Central / PostgreSQL)

Diagram 5 — Device Management Entity-Relationship Diagram (Central / PostgreSQL)

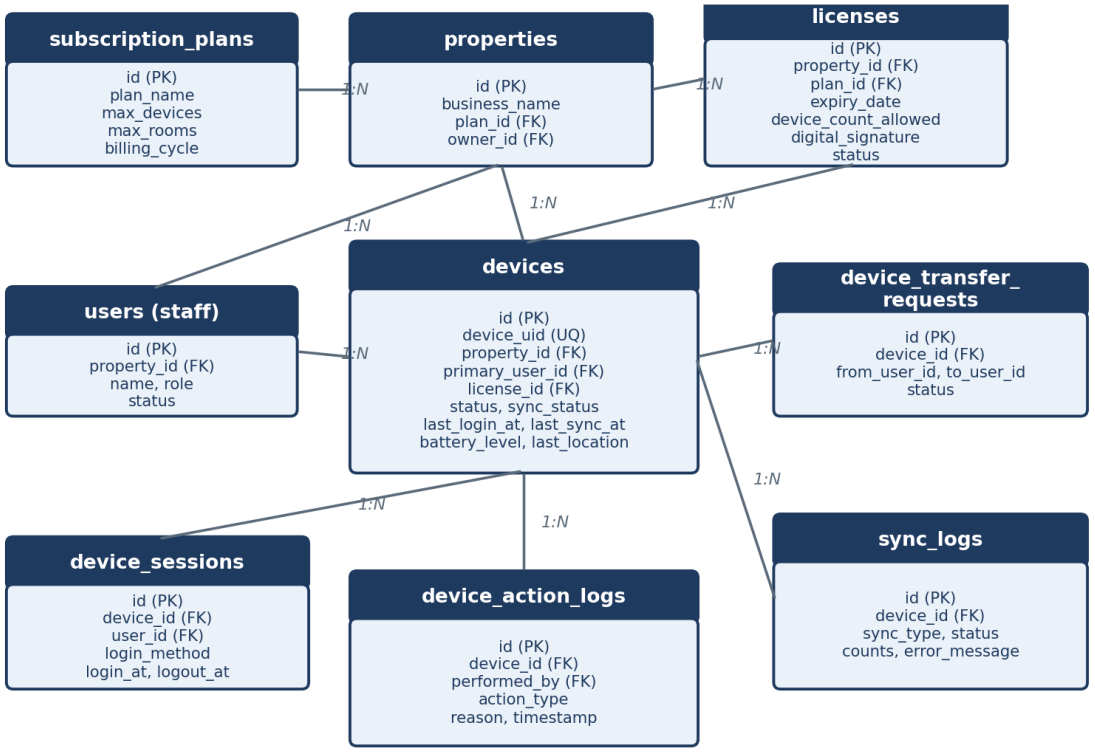

Diagram 5 — Core tables owned or referenced by Device Management.

5.2 Central Schema (PostgreSQL) — Tables, Attributes, Constraints

devices

The core registry entity. One row per physical device ever registered against a property.

Column	Type	Constraints / Notes
id	BIGSERIAL	PRIMARY KEY
device_uid	VARCHAR(128)	UNIQUE, NOT NULL — hashed Android ID / IMEI
property_id	BIGINT	FK → properties.id, NOT NULL, ON DELETE CASCADE
primary_user_id	BIGINT	FK → users.id, NULLABLE (unassigned until first login)
license_id	BIGINT	FK → licenses.id, NULLABLE until approved
device_name	VARCHAR(80)	Owner-editable label (Rename action), NOT NULL, default = model name
device_model	VARCHAR(80)	e.g. ‘Samsung M35’, captured at registration
os_type	VARCHAR(20)	NOT NULL, default 'android'
os_version	VARCHAR(20)	e.g. ‘Android 14’
app_version	VARCHAR(20)	Installed app build version
status	VARCHAR(20)	NOT NULL, CHECK IN (pending_approval, active, rejected, locked, disabled, revoked)
approval_status	VARCHAR(15)	CHECK IN (pending, approved, rejected), default 'pending'
approved_by	BIGINT	FK → users.id, NULLABLE
approved_at	TIMESTAMPTZ	NULLABLE
is_primary_device	BOOLEAN	NOT NULL, default TRUE — flags the property's main registered device
last_login_at	TIMESTAMPTZ	NULLABLE
last_sync_at	TIMESTAMPTZ	NULLABLE, indexed — drives Offline detection
sync_status	VARCHAR(15)	CHECK IN (synced, pending, failed), default 'pending'
battery_level	SMALLINT	0–100, NULLABLE, last reported
last_lat	NUMERIC(9,6)	NULLABLE
last_lng	NUMERIC(9,6)	NULLABLE
registered_at	TIMESTAMPTZ	NOT NULL, default now()
created_at / updated_at	TIMESTAMPTZ	NOT NULL, default now()

Indexes: (property_id), (device_uid) UNIQUE, (last_sync_at) for offline-detection sweeps.

licenses

Represents an issued offline license token, one row per issuance (history preserved).

Column	Type	Constraints / Notes
id	BIGSERIAL	PRIMARY KEY
property_id	BIGINT	FK → properties.id, NOT NULL
plan_id	BIGINT	FK → subscription_plans.id, NOT NULL

Column	Type	Constraints / Notes
device_id	BIGINT	FK → devices.id, NULLABLE — set once bound to a device
license_code	VARCHAR(64)	UNIQUE, NOT NULL — public-facing License ID
expiry_date	DATE	NOT NULL
device_count_allowed	SMALLINT	NOT NULL — snapshot of plan.max_devices at issue time
digital_signature	TEXT	NOT NULL — base64 signature blob
token_payload	TEXT	NOT NULL — encrypted token issued to the device
status	VARCHAR(15)	CHECK IN (active, expired, revoked, superseded)
issued_at	TIMESTAMPTZ	NOT NULL, default now()
revoked_at	TIMESTAMPTZ	NULLABLE

Partial unique index: only one license per (property_id, device_id) may have status='active' at a time.

device_sessions

Column	Type	Constraints / Notes
id	BIGSERIAL	PRIMARY KEY
device_id	BIGINT	FK → devices.id, NOT NULL, ON DELETE CASCADE
user_id	BIGINT	FK → users.id, NOT NULL
login_method	VARCHAR(15)	CHECK IN (biometric, pin, password)
login_at	TIMESTAMPTZ	NOT NULL, default now()
logout_at	TIMESTAMPTZ	NULLABLE
forced_logout	BOOLEAN	NOT NULL, default FALSE — true if ended via remote Logout/Revoke action

device_action_logs

Append-only. Mirrors into the platform Audit Log; API layer denies UPDATE/DELETE on this table.

Column	Type	Constraints / Notes
id	BIGSERIAL	PRIMARY KEY
device_id	BIGINT	FK → devices.id, NOT NULL
performed_by	BIGINT	FK → users.id, NOT NULL (Super Admin or Owner user)
performed_by_role	VARCHAR(20)	NOT NULL, denormalised for fast filtering
action_type	VARCHAR(20)	CHECK IN (approve, reject, lock, unlock, logout, disable, enable, rename, transfer, revoke)
reason	TEXT	NULLABLE — required in UI for revoke/reject
created_at	TIMESTAMPTZ	NOT NULL, default now(), immutable

sync_logs

Column	Type	Constraints / Notes
id	BIGSERIAL	PRIMARY KEY
device_id	BIGINT	FK → devices.id, NOT NULL, ON DELETE CASCADE
sync_type	VARCHAR(15)	CHECK IN (full, incremental)
status	VARCHAR(15)	CHECK IN (success, partial, failed)
records_pushed	INTEGER	default 0
records_pulled	INTEGER	default 0
conflict_count	INTEGER	default 0
error_message	TEXT	NULLABLE
retry_count	SMALLINT	default 0
started_at / completed_at	TIMESTAMPTZ	started_at NOT NULL; completed_at NULLABLE while in-flight

Indexes: (device_id, started_at DESC) for the device timeline widget.

device_transfer_requests

Column	Type	Constraints / Notes
id	BIGSERIAL	PRIMARY KEY
device_id	BIGINT	FK → devices.id, NOT NULL
from_user_id	BIGINT	FK → users.id, NULLABLE
to_user_id	BIGINT	FK → users.id, NOT NULL
requested_by	BIGINT	FK → users.id, NOT NULL
status	VARCHAR(15)	CHECK IN (pending, approved, rejected), default 'pending'
requested_at / resolved_at	TIMESTAMPTZ	requested_at NOT NULL; resolved_at NULLABLE

*Referenced (owned by other modules, included here only for FK context): **properties**(id, business_name, owner_id, plan_id, ...) — Property Onboarding; **subscription_plans**(id, plan_name, max_devices, max_rooms, billing_cycle, ...) — Subscription Management; **users**(id, property_id, name, role, status, ...) — Staff / User & Role Management.*

5.3 On-Device Local Schema (SQLite / ObjectBox)

Per the BRD, the mobile app's local database primarily holds the operational business data (Rooms, Bookings, Guests, Payments, etc.) so it can run entirely offline. For Device Management specifically, the device only needs to know about *itself* — it has no reason to hold the global device directory. A single local table/box is sufficient:

Field	Type	Purpose
`device_uid`	TEXT	This device's own generated UID — recomputed at boot and compared for tamper detection
`property_id`	TEXT	Bound property, embedded in the signed token
`license_token`	BLOB (encrypted)	The full signed token as issued by the server
`license_expiry`	DATE	Extracted from the token for quick local checks
`digital_signature`	TEXT	Stored separately for signature verification
`local_status`	TEXT	Derived: active / grace / suspended — recomputed on each app start
`grace_period_end`	DATE	Computed as expiry + 7 days (per BRD Grace Period rule)
`last_sync_timestamp`	DATETIME	Used by the app itself to show 'Last synced: ...' to the user
`pending_action_flag`	TEXT	Set when a queued remote command (logout / revoke) is detected on next sync

ObjectBox is a NoSQL object store, so this maps to a single @Entity class with no joins; on plain SQLite it is one table with a single row, keyed by device_uid.

5.4 Relationships & Referential Rules Summary

- properties (1) — (N) devices — a property can register up to its plan's max_devices.
- properties (1) — (N) licenses — full history retained even after renewal/revocation.
- devices (1) — (N) device_sessions, sync_logs, device_action_logs, device_transfer_requests.
- devices (N) — (1) users, via primary_user_id — a user may have historically used more than one device, but a device has exactly one *current* primary user at a time.
- licenses (N) — (1) subscription_plans — snapshotting device_count_allowed avoids retroactive breakage if a plan's limits change later.
- ON DELETE rules: devices cascade-delete their sessions/sync logs if a property is fully deleted (rare, only via Super Admin's permanent delete after retention period); action logs are **never** cascade-deleted — they are retained independently for audit purposes even if the parent device row is archived.

6. Role-Based Access Matrix

Extending the BRD's own Recommended Role Matrix pattern specifically for the Device Management module:

Capability	Super Admin	Owner	Manager	Reception / Others
View device directory	Global (all properties)	Own property	Own property (view)	—
Approve / Reject device	✓	✓ (own property)	—	—
Lock / Unlock / Disable	✓	✓ (own property)	—	—
Force Logout	✓	✓ (own property)	—	—
Rename device	✓	✓ (own property)	—	—
Transfer device to new staff	✓	✓ (own property)	—	—
Revoke License	✓	✓ (own property, with confirmation)	—	—
Force Sync (diagnostic)	✓	✓ (own device)	—	own device only
View sync/device analytics	Global	Own property	Own property	—
View device action / audit logs	Global	Own property	—	—

The Support Portal role is intentionally handled separately in Section 7.3: it can view diagnostics and force a sync, but per the BRD's Support Portal feature list it does not appear alongside Revoke License / Approve, which remain owner/subscription-tier decisions.

7. Dashboards

The BRD's device-management responsibilities surface through **three** distinct dashboards/panels, each scoped to a different persona and each pulling from the same central schema in Section 5:

7.1 Super Admin — Global Device Management Console

Use case: Pinesphere Operations monitors and controls every registered device across every tenant property — approving new devices, enforcing plan device-limits, and shutting off compromised or resigned-staff devices platform-wide.

Summary cards

- Total Registered Devices
- Pending Approvals
- Active Devices
- Locked / Disabled Devices
- Offline Devices (no sync > 24h)
- Failed Syncs (last 24h)

Device directory table — fields

- Device ID / UID

- Device Model & Name
- Property Name
- Owner Name
- Current (Primary) User
- OS Version
- App Version
- Status (pending / active / locked / disabled / revoked)
- Approval Status
- Last Login
- Last Sync
- Sync Status
- Battery %
- Last Known Location
- Registered Date

Filters & actions

- Filters: Property, Status, Sync Status, Subscription Plan, Date range.
- Row actions: Approve, Reject, Lock, Unlock, Logout, Disable, Enable, Rename, Transfer, Revoke License, Force Sync.
- Detail drawer: device action timeline, sync history chart, masked license token details, session history.

7.2 Property Owner — My Devices Panel

Use case: the property owner views and manages the phones/tablets used by their own staff — approving a new staff member's device, watching sync health, and revoking access the moment someone leaves.

Summary cards

- My Devices (e.g. “3 / 5 used” against plan limit)
- Pending Approval
- Active
- Offline

Device list — fields

- Device Name
- Assigned Staff & Role
- Status
- Last Login
- Last Sync
- Battery %
- Approval Status

Actions & alerts

- Approve / Reject a new staff device request.
- Lock, Force Logout, Rename, Transfer to another staff member.
- Revoke License (with a confirmation step, since it erases the device's local data on reconnect).
- “Request device limit upgrade” deep link into Subscription Management when at plan ceiling.
- Alerts panel: e.g. “Reception tablet hasn't synced in 2 days”, “License expires in 5 days”.

7.3 Support Engineer — Device Diagnostics Panel

Use case: Pinesphere's Support team troubleshoots a customer-reported sync/device issue without holding the authority to revoke licenses or approve devices — matching the BRD's Support Portal feature list (Force Sync, Remote Diagnostics, View Sync Status, View Errors).

Fields & actions

- Search by Device ID / Property / Customer mobile number.
- View: current sync status, last 10 sync attempts with error messages, connectivity/session logs, OS & app version.
- Actions: Force Sync, Reset License (re-issue token), Generate Diagnostic Report, Chat with Customer.
- No access to: Approve/Reject, Revoke License, subscription/plan changes.

8. API Surface (Indicative)

A representative FastAPI route set for this module — exact paths/naming to be finalised against the team's existing SIMP-style backend folder structure conventions.

Endpoint	Method	Purpose
/devices/register	POST	First-time device registration (requires internet)
/devices/{id}/activate	POST	Issue license token after approval
/devices	GET	List devices (scoped by role: global for Super Admin, property-scoped for Owner)
/devices/{id}	GET	Device detail incl. session & sync history
/devices/{id}/approve	POST	Approve pending device (checks plan device-count limit)
/devices/{id}/reject	POST	Reject pending device
/devices/{id}/lock	POST	Temporary lock
/devices/{id}/unlock	POST	Remove lock
/devices/{id}/disable	POST	Soft, longer-term disable
/devices/{id}/enable	POST	Re-enable a disabled device
/devices/{id}/logout	POST	Force-end current session
/devices/{id}/rename	PATCH	Update display name
/devices/{id}/transfer	POST	Create a device_transfer_request to a new staff user
/devices/{id}/revoke	POST	Revoke license; queues erase + login-disable for next reconnect
/devices/{id}/force-sync	POST	Support/Owner-triggered immediate sync attempt
/devices/sync-checkin	POST	Called by the app itself on every sync cycle; also delivers queued remote commands
/devices/{id}/logs	GET	device_action_logs for this device
/devices/{id}/sync-logs	GET	sync_logs history for this device

9. Implementation Plan (Phase-wise)

Sequenced by technical dependency, not by calendar date — each phase assumes the previous one is functionally complete.

Phase 1 — Foundation

- Design and migrate the central PostgreSQL schema: devices, licenses, device_sessions, device_action_logs, sync_logs, device_transfer_requests.
- Define FastAPI Pydantic schemas and SQLAlchemy models; wire foreign keys to properties, users, subscription_plans owned by other modules.
- Define the on-device local schema (SQLite table / ObjectBox entity) for license storage.

Phase 2 — Registration & Activation

- Build /devices/register and /devices/{id}/activate endpoints, including Device UID generation strategy on the Flutter side.
- Implement the license-token signing service (asymmetric signature, e.g. RSA/ECDSA) and the on-device verification routine.
- Build the Flutter onboarding screen: Property ID + Activation Code → pending-approval state.

Phase 3 — Approval Workflow & Role Enforcement

- Build device-count enforcement against subscription_plans.max_devices at approval time.
- Implement role-scoped listing/authorization middleware (Super Admin: global; Owner: own property only).
- Build the Approve/Reject actions and their notifications.

Phase 4 — Device Actions & Remote Control

- Implement Lock/Unlock, Disable/Enable, Logout, Rename endpoints and their local-app enforcement handlers.
- Implement device_transfer_requests flow (request → approval → primary_user_id swap).
- Implement Revoke License + the sync-checkin-based command delivery + local data erase routine on the Flutter side.

Phase 5 — Sync Engine Integration

- Wire the background sync service to write sync_logs and update devices.last_sync_at / sync_status.
- Implement incremental sync, conflict resolution hooks, and auto-retry with exponential backoff.
- Implement the Offline-detection sweep (scheduled job) that flags devices past the sync threshold.

Phase 6 — Dashboards & Notifications

- Build the Super Admin Global Device Management Console (Section 7.1).
- Build the Owner's My Devices panel (Section 7.2).
- Build the Support Engineer Diagnostics panel (Section 7.3).
- Wire Notification Center triggers: Device Offline, Sync Failure, Pending Approval, License Expiry Warning.

Phase 7 — Security Hardening

- Harden digital-signature verification against tampering/replay; pin the public key in the app binary.
- Encrypt local license storage at rest (Flutter secure storage) and add biometric/PIN gating on app entry.
- Enforce append-only permissions on device_action_logs at the database role level.
- Test device-spoofing scenarios: factory reset, Device UID collision, SIM swap, cloned APK.

Phase 8 — Testing & Documentation

- Integration tests across online/offline/grace/suspended transitions.
- Load-test the device directory queries at scale (multi-tenant, thousands of devices).
- Edge cases: multiple devices per property at the plan limit, transfer mid-sync, revoke while offline.
- Handover documentation for the Support team (diagnostics runbook) and Owner-facing help content.

10. Security, Scalability & Extensibility Notes

Security

- License tokens are signed (not just encrypted) so tampering is detectable without a server round-trip, directly implementing the BRD's “Digital Signature” requirement.
- device_action_logs is append-only at the database-role level (no UPDATE/DELETE grants for the API service account) so audit history cannot be altered even by a compromised backend credential.
- Device UID should combine multiple hardware signals (not just Android ID alone, which can reset on factory reset) to reduce spoofing/re-registration abuse.
- Revoke must be idempotent and re-checked on every sync-checkin, since push delivery to an offline device cannot be guaranteed.

Scalability

- All device tables are naturally partitionable by property_id, which supports future sharding as tenant count grows.
- Indexes on (property_id), (device_uid), and (last_sync_at) keep the Super Admin's global dashboard and the offline-detection sweep performant at scale.
- The offline-detection sweep and auto-retry logic should run as Celery background jobs (per the BRD's stated backend stack) rather than inline on API requests.

Extensibility

- action_type and status values are modelled as CHECK-constrained enums now, but should migrate to a small lookup table (device_action_types) if new action types are added frequently post-launch, without a schema migration each time.
- device_count_allowed is snapshotted on the license row (not read live from subscription_plans) so future plan-limit changes never silently break already-issued licenses.
- The three dashboards share one underlying devices query layer with role-based scoping applied at the service layer, so a fourth persona (e.g. a future Regional Manager role already shown in the BRD's Super Admin hierarchy) can be added by adding a new scope rule, not a new data model.

End of document — Device Management Module Blueprint, derived from the Pinesphere Stay Business Requirement Document.